

Big-picture

We want to build an agent that can learn how good a game state is.

Instead of writing fixed rules manually, we define some **features** for a state, and the agent learns a **weight** for each feature.

Then the value of a state is estimated using:

$$\langle \hat{V}(x) = w_1x_1 + w_2x_2 + \dots + w_nx_n \rangle$$

If a state leads to a good future, its value should become larger.

If it leads to losing, its value should become smaller.

2. What you must build

implement a Snake agent on a **20×20 board** using **linear value-function approximation**.

Required elements

- Board size: **20×20**
 - Snake starts with length **3**
 - There are **four random 2×2 obstacle blocks**
 - There is **one random food**
 - Start from a **random board state**
 - In each non-terminal state, choose the action whose **next state has the highest estimated value**
 - If terminal:
 - collision with self: $V_{\text{train}}(s_t) = -1000$
 - collision with obstacle: $V_{\text{train}}(s_t) = -1000$
 - Use the weight update rule with a **small learning rate α**
-

you must implement the **same learning idea** for Tic-tac-toe.

- Win: $V_{\text{train}}(s_t) = 100$
 - Lose: $V_{\text{train}}(s_t) = -100$
 - Draw: $V_{\text{train}}(s_t) = 0$
-

3. Core concept

The central ML idea is:

We do not store value for every possible state

Because there are too many states.

Instead, we:

1. Represent each state by a **feature vector** $x = [x_1, x_2, \dots, x_n]$
2. Learn weights $w = [w_1, w_2, \dots, w_n]$
3. Estimate state value: $\hat{V}(x) = w^T x$

This is a **linear approximation**.

4. Implementation Steps

Step 1 — Represent the Snake state

A Snake state includes things like:

- snake head position
- snake body positions
- food position
- obstacles
- current direction
- whether nearby cells are dangerous

But we do not directly feed the full board into a simple linear model.

Instead, we create **features**.

Step 2 — Design features for Snake

This is one of the most important parts.

Example Snake features

a feature should describe something useful about the state.

Here is a good beginner feature set: $X = [X_1, X_2, X_3, X_4, X_5, X_6, X_7]$

Where:

- x_1 : bias term = 1
- x_2 : food is above the snake head
- x_3 : food is below the snake head
- x_4 : food is left of the snake head
- x_5 : food is right of the snake head
- x_6 : danger straight ahead
- x_7 : distance to food

You can also add:

- danger left
 - danger right
 - number of free neighboring cells
 - normalized Manhattan distance to food
 - whether moving closer to food
-

feature example

Suppose:

- head = (10,10)
- food = (10,14)
- snake is moving right
- cell in front is safe
- left is blocked
- right is safe

Then:

- bias = 1
- food above? no = 0
- food below? no = 0
- food left? no = 0
- food right? yes = 1
- danger straight? no = 0
- distance to food = 4

So the feature vector may be:

$$x=[1,0,0,0,1,0,4]$$

If the weights are:

$$w=[2,-1,-1,-1,3,-5,-0.5]$$

Then:

$$\hat{V}(x)=2(1)+(-1)(0)+(-1)(0)+(-1)(0)+3(1)+(-5)(0)+(-0.5)(4)$$

$$\hat{V}(x)=2+3-2=3$$

So the estimated value of that state is **3**.

Step 3 — Initialize weights

Initialize all weights to **zero**.

So at the beginning:

$w = [0, 0, 0, \dots, 0]$

This means initially the agent has no knowledge.

Step 4 — Generate the initial Snake board

The starting board should have:

- size **20×20**
- snake length **3**
- **4 random 2×2 obstacles**
- **1 random food position**

Example

- snake body: (5,5), (5,4), (5,3)
- food: (12,15)
- obstacles:
 - block 1 at (2,2), (2,3), (3,2), (3,3)
 - block 2 at (8,8), (8,9), (9,8), (9,9)
 - block 3 at (14,4), (14,5), (15,4), (15,5)
 - block 4 at (17,16), (17,17), (18,16), (18,17)

Need to ensure **no overlap** between:

- snake
- food

- obstacles
-

Step 5 — Evaluate possible actions

At each step, the snake can usually choose among actions like:

- up
- down
- left
- right

For each possible action:

1. Compute the **next state**
2. Extract its features
3. Estimate its value using: $\hat{V}(x) = \hat{w}^T \vec{x}$
4. Choose the action with the **highest estimated value**

This is exactly the greedy behavior:

choose the move whose successor has the highest estimated value.

Example of action selection

Suppose current state is s_t , and there are 3 legal moves:

- Up leads to s_1
- Left leads to s_2
- Right leads to s_3

Suppose:

- $\hat{V}(s_1) = 5$
- $\hat{V}(s_2) = 2$

- $\hat{V}(s_3) = 8$
- Then the agent chooses **Right**, because:

$$\max(5, 2, 8) = 8$$

So:

$$S_{t+1} = s_3$$

Then for a non-terminal state:

$$V_{\text{train}}(s_t) = \hat{V}(s_{t+1})$$

$$V_{\text{train}}(s_t) = 8$$

Terminal states

- if snake hits itself: $V_{\text{train}}(s_t) = -1000$
- if snake hits obstacle: $V_{\text{train}}(s_t) = -1000$

Also the snake loses by hitting the border, so border collision should also be treated as terminal in implementation.

This is a very strong negative signal, telling the agent:

never choose states that lead to dying.

Step 6 — Weight update

$$w_i = w_i + \alpha (V_{\text{train}}(s_t) - \hat{V}(s_t)) x_i$$

This is the learning rule.

Meaning of each part

- w_i : current weight
- α : learning rate
- $V_{\text{train}}(s_t)$: target value

- $\hat{V}(s_t)$: current predicted value
- error: $V_{\text{train}}(s_t) - \hat{V}(s_t)$

If prediction is too low, weights move up.

If prediction is too high, weights move down.

Full numeric example of weight update

Suppose:

- $\alpha = 0.1$
- current features: $x = [1, 0, 1]$
- current weights: $w = [0.5, 0.2, -0.1]$

Then predicted value is:

$$\hat{V}(x) = 0.5(1) + 0.2(0) + (-0.1)(1) = 0.4$$

Suppose next state's estimated value is 2.0, so:

$$V_{\text{train}}(s_t) = 2.0$$

Error:

$$2.0 - 0.4 = 1.6$$

Now update each weight:

$$w_1 = 0.5 + 0.1(1.6)(1) = 0.66$$

$$w_2 = 0.2 + 0.1(1.6)(0) = 0.2$$

$$w_3 = -0.1 + 0.1(1.6)(1) = 0.06$$

New weights:

$$w = [0.66, 0.2, 0.06]$$

This means the model now gives a higher value to similar states.

5. Full Snake training loop explanation

Training algorithm for Snake

1. Initialize weights to zero
 2. Create random board
 3. While state is not terminal:
 - list legal moves
 - compute successor state for each move
 - compute features for each successor
 - estimate value of each successor
 - choose the move with highest estimated value
 - if next state is terminal:
 - assign -1000
 - else:
 - set
- $$V_{\text{train}}(s_t) = \hat{V}(s_{t+1})$$
- update weights using the rule
 - move to next state
4. Repeat for many episodes
5. After training, use learned weights to play